```
template<typename D>
class LIST {
  node<D>* H;
  node<D>* T;
  long int elem;

public:

  class iterator {
  node<D>* ptr;

 public:
   iterator(node<D>* p = nullptr) : ptr(p) {}

   D& operator*(){
     if(!ptr) throw std::runtime_error("Null iterator");
     return ptr->K;
   }
   const D& operator*() const {
     if(!ptr) throw std::runtime_error("Null iterator");
     return ptr->K;
   }

   iterator& operator++() noexcept {
     if(ptr) ptr = ptr->P;
     return *this;
   }
   iterator operator++(int) noexcept {
     iterator temp = *this;
     ++(*this);
     return temp;
   }

   bool operator!=(const iterator& other) const noexcept { return ptr != other.ptr; }
   bool operator==(const iterator& other) const noexcept { return ptr == other.ptr; }

   D* operator->(){ return &(ptr->K); }
   const D* operator->() const { return &(ptr->K); }

        // string s = "hello"; string* p = &s;
        // cout << p->size();  we cannot overload ( . ) in C++
 };                              // thats why Overload ( -> )
```

// “Iterator class is needed to wrap raw pointers and provide controlled access with operator overloading so the container behaves like STL instead of exposing internal structure.”

// “We use iterator instead of returning raw pointer because pointer exposes internal node structure and cannot be controlled, while iterator provides abstraction and operator-based access.”

```
  iterator begin(){ return iterator(H); }
   iterator end(){ return iterator(nullptr); }

   iterator begin() const{ return iterator(H); }
   iterator end() const{ return iterator(nullptr); }

};



int main(){
   LIST<int> l;

   l.insert(10);
   l.insert(20);
   l.insert(30);

   for(auto it = l.begin(); it != l.end(); ++it){
      cout << *it << " ";
   }

   for(auto x : l){
      cout << x << " ";
   }
         // for(auto x : l)  👉 Compiler converts it into
         // auto && __range = l;
         // for(auto __it = __range.begin(); __it != __range.end(); ++__it){
         //     auto x = *__it;
         // }
}
```